

PROJECT BASE CASE

ASSIGNMENT & PROJECTS 03 & 04

TABLE OF CONTENTS

Assignment 03	1
Project 03.....	2
Scenario	2
Requirements.....	2
Attributes	2
Constructor.....	2
Setters (with validation).....	2
A method to calculate the item total	2
Magic Method: toString().....	2
Test it in main()	3
Expected Output	3
Assignment 04	3
Project 04.....	3
Objective	3
The Scenario	3
Requirements.....	3
Expected Output	4
NOTES	5

ASSIGNMENT 03

1. Classes & Objects: [<https://www.youtube.com/watch?v=DYbi93vuSaU>]
2. Attributes/Properties [<https://www.youtube.com/watch?v=IUqKuGNasdM>]
3. Methods (private & public) – for now, shudhu basic public privat ebujhlei hobe. Chaile details ebujhte paro. Inheritance korar time e shob emnio clear hoye jabe. [<https://www.youtube.com/watch?v=mlVYooy93sE&t=190s>]
4. Constructors [<https://www.youtube.com/watch?v=ZD7CB6wKg8A>]
5. Magic Method: toString() [<https://www.youtube.com/watch?v=GvbdMwfjB98>]
6. Getters and Setters [https://www.youtube.com/watch?v=OjrR_C_UPjc]
7. Basic Exception throw
8. Handling [<https://www.youtube.com/watch?v=adTDlH0lhaA>]
9. Throw [<https://www.youtube.com/watch?v=kAOYeVYtukY>]

PROJECT 03

SCENARIO

The shop's checkout system is growing. Instead of scattering product data across multiple variables, the owner wants each order to be a self-contained unit that knows its own details and can calculate its own total. Your job is to build that unit properly, with clean validation so that no corrupt data ever enters the system.

Task

Build a class called `Order` that represents a single product order in a shop.

REQUIREMENTS

The `Order` class must have:

ATTRIBUTES

- A product name (`String`) - this can be public or private, your choice.
- A unit price (`double`) - must be private.
- A quantity (`int`) - must be private.

CONSTRUCTOR

Takes three parameters: name, unit price, and quantity.

Before assigning unit price and quantity, validate them. If either value is zero or negative, throw an exception with a clear message explaining what went wrong.

Hint: `throw new Exception("your message here");` — and your constructor signature will need `throws Exception`.

SETTERS (WITH VALIDATION)

Write a `setUnitPrice(double price)` method and a `setQuantity(int qty)` method.

Each setter must apply the same validation as the constructor. Do not allow invalid values (in this case quantity can't be 0 or less & unit_price can't be less than 0) through silently.

A setter is a public method whose only job is to safely update a private field.

A METHOD TO CALCULATE THE ITEM TOTAL

Returns `unit_price × quantity` as a `double`. This is a public method.

MAGIC METHOD: `toString()`

Override this method to return a formatted string representing the order.

Format: ProductName - Qty pcs x UnitPrice = Total

Example: Gaming Mouse - 3 pcs x 25.75 = 77.25

TEST IT IN MAIN()

Create at least two valid Order objects and print them.

Attempt to create one with a negative price or zero quantity inside a try-catch block and print the error message.

EXPECTED OUTPUT

```
--- Order Test ---
Gaming Mouse - 3 pcs x 25.75 = 77.25
Mechanical Keyboard - 1 pcs x 120.0 = 120.0
--- Invalid Order Attempt ---
Error: Unit price must be greater than zero.
```

ASSIGNMENT 04

1. ArrayList [<https://www.youtube.com/watch?v=NbYgm0r7u6o>]
2. Foreach Loop [<https://www.youtube.com/watch?v=KcJX6XGmB-o>]
[<https://www.youtube.com/watch?v=IT8F5W0ZO4>]

PROJECT 04

OBJECTIVE

Apply what you learned in **Assignment 03** to build a standalone, well-structured Order class — the core building block that your next project will plug into.

THE SCENARIO

The shop's system is getting smarter. You now have a proper Order class that validates its own data. The next step is to use it inside a real checkout flow — replacing the old parallel arrays with a clean `ArrayList<Order>` and making the whole system more robust.

REQUIREMENTS

The calculations and logics are the same as project 02. Now the changes shall be:

Use Order class and `ArrayList<Order>` to manage the work with proper exception handling.

STEP 1 — ASK FOR NUMBER OF ITEMS

Ask the user how many items they want to purchase. Cap it at 5 using an `if-else`. If they enter more than 5, set it to 5 and inform them.

STEP 2 — COLLECT ORDERS IN A LOOP

Use a `for` loop to collect each item's name, price, and quantity from the user.

For each item, attempt to create a new `Order` object inside a `try-catch` block. If the `Order` constructor throws an exception (invalid price or quantity), display the error, then **do not add that item** to the list, and **do not count it** against the item total — the user effectively skips that slot.

Add valid orders to an `ArrayList<Order>`.

STEP 3 — CALCULATE THE SUBTOTAL

Use a `for-each` loop over your `ArrayList<Order>`. For each order, call `calculateTotal()` and accumulate the subtotal.

STEP 4 — APPLY DISCOUNT LOGIC

Using `if-else`:

- Subtotal above 100 → 20% discount
 - Subtotal above 50 → 10% discount
 - Otherwise → 5% discount
-

STEP 5 — DELIVERY FEE

Using a ternary operator:

- If subtotal is below 50 → delivery fee is 10
 - Otherwise → delivery fee is 0
-

STEP 6 — CALCULATE FINAL TOTAL

```
finalTotal = subtotal - discountAmount + deliveryFee
```

STEP 7 — PRINT THE RECEIPT

Use a `for-each` loop to print each order. Since your `Order` class has `toString()`, printing the object directly will produce the correct format. Then print the summary.

EXPECTED OUTPUT

```
--- Welcome to BaseCase Shop ---
```

How many items do you want to purchase? 3

Enter Product 1 Name: Mouse

Enter Price: 500

Enter Quantity: 2

Enter Product 2 Name: Keyboard

Enter Price: -50

Enter Quantity: 1

Error: Unit price must be greater than zero. Skipping item.

Enter Product 3 Name: USB Hub

Enter Price: 150

Enter Quantity: 1

--- Final Receipt ---

Item 1: Mouse - 2 pcs x 500.0 = 1000.0

Item 2: USB Hub - 1 pcs x 150.0 = 150.0

Subtotal: 1150.0

Discount Applied (20%): 230.0

Delivery Fee: 0.0

Final Total: 920.0

NOTES

Assignment 03: 15pts

Project 03: 20pts

Assignment 04: 10pts

Project 04: 20pts

For extra creativity: 10pts

[You are allowed to apply improvements or any other techniques you want to. These are minimum requirements]

[If you do not understand any part of the project or have any confusion, feel free to ask right away]